

Kubernetes cvičná úloha — kompletný návod

Krok-za-krokom, čo znamená každý riadok

Contents

Úvod — ako čítať tento návod	2
1. Štruktúra projektu	2
2. Dockerfile — recept na image	3
3. Build image-u	3
A) Lokálne do Minikube (najjednoduchšie pre obhajobu)	3
B) Push na Docker Hub (alternatíva — image v cloude)	3
4. Kubernetes súbory (k8s/) — postupne	4
4.1 namespace.yaml	4
4.2 configmap.yaml	4
4.3 pv.yaml — PersistentVolume (úložisko)	4
4.4 pvc.yaml — PersistentVolumeClaim (žiadosť o úložisko)	5
4.5 deployment.yaml — pod a všetko okolo neho	5
4.6 service.yaml — vystavenie aplikácie	6
5. Porty — najväčší zdroj zmätku	7
Tabuľka — kde čo je	7
Čo sedí s čím (tu sa najčastejšie chybí)	7
Otvorenie aplikácie — 2 spôsoby	7
Príklad: Apache na 8080 namiesto 80	8
6. Viac ConfigMáp	8
Príklad — appka má backend a UI nastavenia	8
Použitie v deployment-e	9
7. Viac Deploymentov (BE + FE)	9
Image-y	9
Deployment BE	10
Deployment FE	10
Service BE — interná, len pre FE	11
Service FE — verejná	11
Komunikácia FE → BE	11
8. Nasadenie a overenie	12
Krok za krokom	12
Cieľový stav (čo musíš vidieť)	12
Otvor v prehliadači	12
Dôkaz perzistencie (povinný pri obhajobe)	12
9. Debug — keď niečo nejde	13
Príkazy v poradí, keď ti nefunguje	13

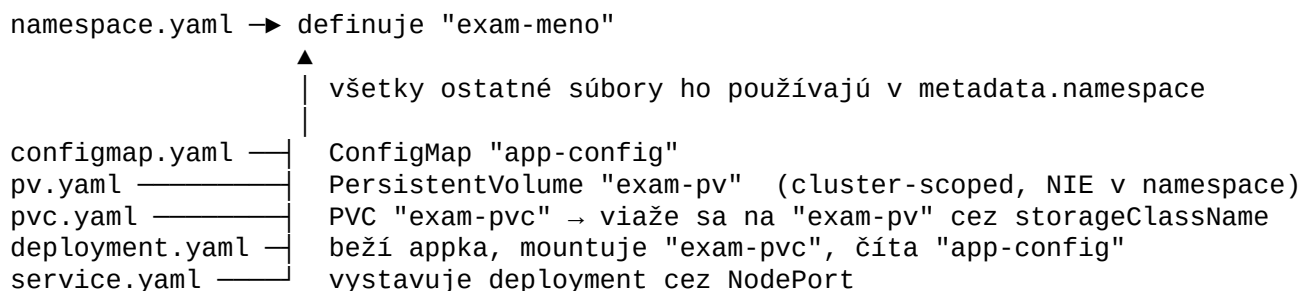
Najčastejšie chyby	13
10. Upratanie	14
11. Cheatsheet — len príkazy v poradí	14
12. Mapa: ktorá hodnota kde a s čím sa zhoduje	15

Úvod — ako čítať tento návod

Cieľ: nasadiť aplikáciu (PHP web) do Kubernetes (Minikube) tak, aby splnila všetky body zadania (60 bodov + bonusy).

Dôležitá vec na začiatok: Kubernetes je len súbor YAML súborov, ktoré popisujú “čo má bežať”. `kubectl apply -f k8s/` to celé prečíta a spustí. Keď niečo nefunguje, najčastejšie ide o to, že **mená sa medzi súbormi nezhodujú** (napr. `claimName` v `deployment-e` ≠ `metadata.name` v `PVC`). Veľmi pozorne kontroluj mená.

Súbory sú prepojené takto:



1. Štruktúra projektu

Vytvor si pracovný priečinok niekde mimo poznámok:

```
mkdir -p ~/exam/{src,k8s}
cd ~/exam
```

Cieľ:

```
~/exam/
├── Dockerfile           ← recept ako postaviť image
├── README.md            ← dokumentácia (povinná, 5 bodov)
├── src/                 ← kód aplikácie
│   ├── index.php        ← hlavný súbor appky
│   ├── health.php       ← bonus: liveness probe
│   └── ready.php        ← bonus: readiness probe
└── k8s/                 ← všetky Kubernetes manifesty
    ├── namespace.yaml
    ├── configmap.yaml
    ├── pv.yaml
    ├── pvc.yaml
    ├── deployment.yaml
    └── service.yaml
```

Prečo dva priečinky? `src/` je kód, ten ide do Docker image-u. `k8s/` sú manifesty, tie ide ako `kubectl apply`. Sú to dve úplne odlišné veci — image je “appka v krabici”, `k8s` je “ako tú krabicu spustiť”.

2. Dockerfile — recept na image

```
FROM php:8.2-apache          # základ: Apache + PHP, počúva na porte 80

COPY src/ /var/www/html/     # nakopíruj náš kód dnu

RUN mkdir -p /var/www/html/data \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80                    # informačné: appka počúva na 80
```

Vysvetlenie:

- FROM php:8.2-apache — oficiálny image z Docker Hubu, **má v sebe Apache + PHP** zlepené cez mod_php. Vďaka tomu nemusíš riešiť nginx + php-fpm.
 - COPY src/ /var/www/html/ — Apache očakáva HTML/PHP v /var/www/html/, takže tam dáme náš kód.
 - RUN ... chown — Apache beží pod userom www-data (UID 33). Aby mohol písať súbory, musí byť ich vlastníkom.
 - EXPOSE 80 — len dokumentačná značka, nič nezapne. Appka tak či tak počúva na 80.
-

3. Build image-u

A) Lokálne do Minikube (najjednoduchšie pre obhajobu)

```
docker build -t exam-app:latest -f Dockerfile .
#          ▲ tag = názov:verzia      ▲ ktorý Dockerfile      ▲ kontext (kde je src/)
```

```
minikube image load exam-app:latest
#          ▲ nakopíruje image z tvojho Dockeru DO Minikube
```

Prečo minikube image load? Minikube má **vlastný oddelený Docker daemon**. Keď postavíš image vo svojom Dockeri, Minikube ho nevidí. Tento príkaz ho preniesie.

V deployment.yaml potom musíš mať:

```
image: exam-app:latest
imagePullPolicy: IfNotPresent      # NETIAHNI z internetu, použi lokálny
```

Bez IfNotPresent by Kubernetes skúsil image stiahnuť z Docker Hubu a hodil chybu ErrImagePull.

B) Push na Docker Hub (alternatíva — image v cloude)

```
# 1. Prihlásenie (raz za stroj)
docker login
# → zadáš username + password (alebo Personal Access Token)

# 2. Postav image S TVOJÍM USERNAMEOM v tagu (Docker Hub to vyžaduje)
docker build -t mojeusername/exam-app:latest -f Dockerfile .

# alebo retag už postaveného:
docker tag exam-app:latest mojeusername/exam-app:latest

# 3. Push do registry
docker push mojeusername/exam-app:latest
```

V deployment.yaml:

```
image: mojeusername/exam-app:latest
imagePullPolicy: Always # vždy stiahni najnovšiu verziu
```

Public repo = ktokoľvek si môže image stiahnuť, bez prihlásenia. Stačí ti toto.

Private repo = potrebuješ povedať Kubernetes prihlasovacie údaje:

```
kubectl -n exam-meno create secret docker-registry regcred \
  --docker-username=mojeusername \
  --docker-password=TOKEN \
  --docker-email=mail@example.com
```

a v `deployment.yaml` pod `spec.template.spec`:

```
imagePullSecrets:
- name: regcred
```

4. Kubernetes súbory (k8s/) — postupne

4.1 namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: exam-meno # ← VYMEŇ za svoje meno, napr. exam-jano
```

Čo je namespace? Logický “priečinok” v klastri. Všetko ostatné (ConfigMap, Deployment, Service...) bude v ňom. Pri kubectl príkazoch musíš dávať `-n exam-meno`, inak ti nič neukáže.

Tip: Vo všetkých YAML súboroch musíš mať `metadata.namespace: exam-meno` rovnako napísané. Najľahšie:

```
sed -i 's/exam-meno/exam-jano/g' k8s/*.yaml
```

4.2 configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config # ← TENTO názov budeš odkazovať v deployment.yaml
  namespace: exam-meno
data:
  APP_NAME: "Exam App" # tieto kľúče sa stanú env premennými v podo
  MAX_UPLOAD_MB: "10"
  LOG_LEVEL: "INFO"
  WELCOME_TEXT: | # | znamená viacriadkový text
    Vitajte v aplikácii!
    Tento text pripravil InitContainer.
```

Načo to je? Aby si nemal konfiguráciu zabezčenú v image-i. Keď zmeníš `MAX_UPLOAD_MB` z 10 na 20, **nemusíš robiť nový image** — stačí `kubectl apply + rollout restart`.

V PHP kóde čítaš premenné cez `getenv('MAX_UPLOAD_MB')`.

4.3 pv.yaml — PersistentVolume (úložisko)

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

```

name: exam-pv          # ← TENTO názov bude PVC hľadať
                        # POZOR: PV NEMÁ namespace (je cluster-scoped)
spec:
  storageClassName: manual # ← MUSÍ byť rovnaké v PVC, inak sa neviažu
  capacity:
    storage: 1Gi          # 1 gigabajt
  accessModes:
    - ReadWriteOnce       # jeden pod môže zapisovať
  hostPath:
    path: "/mnt/data/exam-meno" # priamo priečinok na hostovi (Minikube VM)

```

Čo je PV? Reálny úložný priestor. hostPath znamená “vezmi priečinok priamo z OS”. Pre cvičnú úlohu úplne stačí.

4.4 pvc.yaml — PersistentVolumeClaim (žiadost’ o úložisko)

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: exam-pvc          # ← TENTO názov budeš odkazovať v deployment.yaml
  namespace: exam-meno
spec:
  storageClassName: manual # ← MUSÍ sa zhodovať s pv.yaml
  accessModes:
    - ReadWriteOnce       # MUSÍ byť rovnaké alebo užšie ako PV
  resources:
    requests:
      storage: 1Gi        # max toľko, koľko má PV

```

PV vs PVC? PV je samotný disk, PVC je “žiadanka, ktorá ho rezervuje”. Bez PVC by si pod nedostal úložisko. Pod používa PVC, nie PV.

Najčastejšia chyba: PVC zostane v stave Pending ak storageClassName v PV a PVC nie sú úplne rovnaké slovo.

4.5 deployment.yaml — pod a všetko okolo neho

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: exam-app          # názov deployment-u
  namespace: exam-meno
spec:
  replicas: 1              # koľko podov bežia paralelne
  selector:
    matchLabels:
      app: exam-app        # ← Service ich nájde podľa tohto label-u
  template:                # popis pod-u (čo sa replikuje)
    metadata:
      labels:
        app: exam-app      # ← MUSÍ sa zhodovať s matchLabels vyššie
    spec:
      volumes:              # ČO sa dá namountovať
        - name: data        # interný názov
          persistentVolumeClaim:
            claimName: exam-pvc # ← MUSÍ sa zhodovať s metadata.name v pvc.yaml
      initContainers:       # bonus: beží PRED hlavným kontajnerom

```

```

- name: init-data
  image: busybox:1.36
  command: ["sh", "-c"]
  args:
    - |
      set -e
      mkdir -p /data
      chown -R 33:33 /data # 33 = www-data v php image
  volumeMounts:
    - name: data # ← rovnaké meno ako v volumes vyššie
      mountPath: /data # KAM v initContaineri sa namountuje

containers: # hlavný kontajner
- name: exam-app
  image: exam-app:latest # ← rovnaké ako pri docker build -t
  imagePullPolicy: IfNotPresent
  ports:
    - containerPort: 80 # POČUŠETE: kde počúva Apache vnútri kontajnera

  envFrom: # všetky kľúče z ConfigMap-u → env premenné
    - configMapRef:
        name: app-config # ← MUSÍ sa zhodovať s metadata.name v configmap.yaml

  volumeMounts:
    - name: data # ← rovnaké meno ako v volumes vyššie
      mountPath: /var/www/html/data # KAM v hlavnom kontajneri

# bonus: probes
readinessProbe:
  httpGet:
    path: /ready.php
    port: 80 # ← rovnaké ako containerPort
  initialDelaySeconds: 3
  periodSeconds: 5
livenessProbe:
  httpGet:
    path: /health.php
    port: 80
  initialDelaySeconds: 10
  periodSeconds: 10

```

Mapa prepojení (toto si zapamätaj):

Riadok v deployment	Musí sa zhodovať s
selector.matchLabels.app: exam-app	template.metadata.labels.app: exam-app
claimName: exam-pvc	metadata.name v pvc.yaml
configMapRef.name: app-config	metadata.name v configmap.yaml
image: exam-app:latest	tag z docker build -t exam-app:latest
volumeMounts.name: data	volumes.name: data (vyššie v tom istom súbore)

4.6 service.yaml — vystavenie aplikácie

```

apiVersion: v1
kind: Service
metadata:
  name: exam-svc # názov služby (DNS meno v klastri)

```

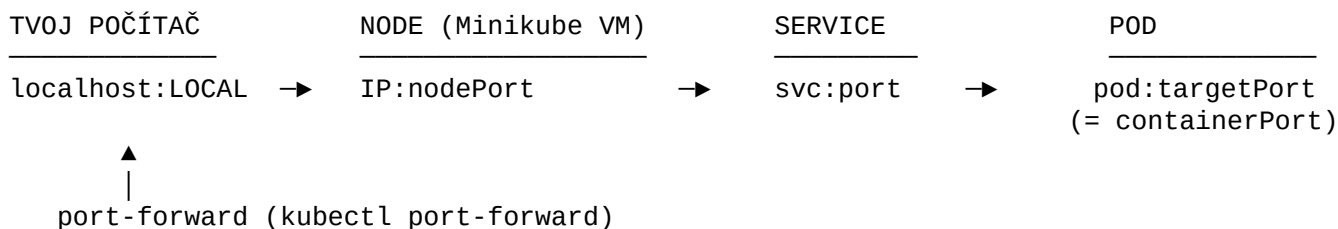
```

namespace: exam-meno
spec:
  selector:
    app: exam-app           # ← MUSÍ sa zhodovať s labels v deployment.yaml
    type: NodePort          # NodePort = dostupné aj zvonku klastra
  ports:
    - port: 80              # port služby v klastri (DNS exam-svc:80)
      targetPort: 80        # ← MUSÍ = containerPort v deployment.yaml
      nodePort: 30080       # port na node-e (zvonku), ROZSAH 30000–32767

```

5. Porty — najväčší zdroj zmätku

Každý port má **inú rolu** a žije na **inej vrstve**. Najprv obrázok:



Tabuľka — kde čo je

Pole	Súbor	Význam	Poznámka
containerPort	deployment.yaml	Kde appka reálne počúva v pode	Apache=80, Node=3000, Python=8000...
targetPort	service.yaml	Kam Service posiela traffic	MUSÍ = containerPort , inak service strelí do prázdna
port	service.yaml	Port Service vnútri klastra	Volíš ľubovoľne. Iné pody volajú exam-svc:port
nodePort	service.yaml	Port navonok	Povinne 30000–32767 . Mimo rozsah K8s zamietne

Čo sedí s čím (tu sa najčastejšie chybí)

```

deployment.yaml: containerPort: 80
                    |
                    |—— MUSÍ byť rovnaké ———|
service.yaml:    targetPort: 80 ←
                  port:      80  ← interný DNS port (ľubovoľne)
                  nodePort: 30080 ← zvonku (30000–32767)

```

Otvorenie aplikácie — 2 spôsoby

1) Cez Minikube tunel (najistejšie na docker-driveri):

```

minikube service exam-svc -n exam-meno
#      ▲ otvorí prehliadač s funkčnou URL

```

2) Cez kubectl port-forward (manuálne):

```
kubectl -n exam-meno port-forward svc/exam-svc 8080:80
#
#               ▲               ▲               ▲
#               service         |         port v service.yaml (`port: 80`)
#               |               |
#               localhost port (čokoľvek čo je voľné)
```

v prehliadači: <http://localhost:8080>

Ako si zapamätať port-forward:

```
kubectl port-forward svc/exam-svc 3001 : 80
#               ▲               ▲
#               tvoj localhost   port služby (`port` v service.yaml)
#               (čokoľvek voľné) (NIE nodePort, NIE targetPort)
```

Príklad: Apache na 8080 namiesto 80

Keby tvoj Apache počúval na 8080 (a nie na 80):

```
# deployment.yaml
ports:
  - containerPort: 8080      # ← appka počúva tu

# service.yaml
ports:
  - port: 80                 # service vystupuje na 80 (môže byť čokoľvek)
    targetPort: 8080         # ← MUSÍ = containerPort = 8080
    nodePort: 30080

kubectl port-forward svc/exam-svc 5331:80
#               ▲ to je `port` zo service.yaml
# → http://localhost:5331
```

6. Viac ConfigMáp

Ak zadanie chce 2+ ConfigMáp, logika je: každá ConfigMap pre svoju oblasť.

Príklad — appka má backend a UI nastavenia

```
# k8s/configmap-app.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config           # PRVÁ ConfigMap
  namespace: exam-meno
data:
  APP_NAME: "Exam App"
  LOG_LEVEL: "INFO"
  MAX_UPLOAD_MB: "10"

# k8s/configmap-ui.yaml
apiVersion: v1
kind: ConfigMap
metadata:
```



```

name: ui-config          # DRUHÁ ConfigMap (iný názov!)
namespace: exam-meno
data:
  SITE_TITLE: "Môj Blog"
  THEME: "dark"
  API_URL: "http://exam-be-svc:80"

```

Použitie v deployment-e

Spôsob 1 — všetky kľúče z oboch ako env premenné:

```

envFrom:
- configMapRef:
  name: app-config      # ← PRVÁ ConfigMap
- configMapRef:
  name: ui-config       # ← DRUHÁ ConfigMap
# v PHP: getenv('APP_NAME'), getenv('SITE_TITLE')

```

Spôsob 2 — vybraný kľúč pod iným menom:

```

env:
- name: BACKEND_URL      # premenná v pode
  valueFrom:
    configMapKeyRef:
      name: ui-config     # z ktorej ConfigMap-y
      key: API_URL        # ktorý kľúč
# v PHP: getenv('BACKEND_URL') → http://exam-be-svc:80

```

Spôsob 3 — ConfigMap ako súbor (napr. welcome.txt):

```

volumes:
- name: cfg              # interný názov volumu
  configMap:
    name: app-config     # z ktorej ConfigMap-y
    items:
      - key: WELCOME_TEXT # ktorý kľúč
        path: welcome.txt # pod akým názvom súboru

# v containers (alebo initContainers):
volumeMounts:
- name: cfg              # ← rovnaké ako volumes.name
  mountPath: /cfg        # → vznikne /cfg/welcome.txt

```

Po zmene ConfigMap-y:

```

kubectl -n exam-meno rollout restart deployment/exam-app
# bez tohto staré pody nepočujú zmenu

```

7. Viac Deploymentov (BE + FE)

Ak zadanie chce **2+ Deployment-y**, najčastejší vzor je rozdelenie na backend a frontend:

- **BE** (backend) = API/logika, mountuje PVC, má svoju ConfigMap, **nedostupný zvonku** (ClusterIP service)
- **FE** (frontend) = UI/web, bez PVC, má svoju ConfigMap s URL na BE, **dostupný zvonku** (NodePort service)

Image-y

Buď 2 Dockerfily, alebo 1 Dockerfile s ARG. Najjasnejšie sú **2 Dockerfily**:

```
docker build -f Dockerfile.be -t exam-be:latest .
docker build -f Dockerfile.fe -t exam-fe:latest .
minikube image load exam-be:latest
minikube image load exam-fe:latest
```

Deployment BE

```
# k8s/deployment-be.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: exam-be           # NÁZOV BE deployment-u
  namespace: exam-meno
spec:
  replicas: 1
  selector:
    matchLabels:
      app: exam-be        # ← LABEL, ktorý hľadá service-be
  template:
    metadata:
      labels:
        app: exam-be      # ← MUSÍ = matchLabels
    spec:
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: exam-pvc      # PVC mountuje len BE
      containers:
        - name: exam-be
          image: exam-be:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: be-config      # ← BE ConfigMap
          volumeMounts:
            - name: data
              mountPath: /var/www/html/data
```

Deployment FE

```
# k8s/deployment-fe.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: exam-fe           # NÁZOV FE deployment-u
  namespace: exam-meno
spec:
  replicas: 1
  selector:
    matchLabels:
      app: exam-fe        # ← INÝ label ako BE!
  template:
    metadata:
      labels:
```

```

    app: exam-fe
spec:
  containers:
    - name: exam-fe
      image: exam-fe:latest
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 80
      envFrom:
        - configMapRef:
            name: fe-config          # ← FE ConfigMap (iná!)
            # bez volumeMounts – FE nepotrebuje PVC

```

Service BE — interná, len pre FE

```

# k8s/service-be.yaml
apiVersion: v1
kind: Service
metadata:
  name: exam-be-svc          # ← TÝMTO menom FE volá BE
  namespace: exam-meno
spec:
  type: ClusterIP           # NEPRÍSTUPNÉ zvonku, len v klastrí
  selector:
    app: exam-be            # ← MUSÍ = label v deployment-be.yaml
  ports:
    - port: 80
      targetPort: 80        # = containerPort BE (80)

```

Service FE — verejná

```

# k8s/service-fe.yaml
apiVersion: v1
kind: Service
metadata:
  name: exam-fe-svc
  namespace: exam-meno
spec:
  type: NodePort            # DOSTUPNÉ zvonku
  selector:
    app: exam-fe            # ← MUSÍ = label v deployment-fe.yaml
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080       # localhost:30080 → FE

```

Komunikácia FE → BE

V FE kóde (PHP/JS) voláš:

`http://exam-be-svc:80/api/...`

▲

Kubernetes DNS — funguje vnútri klastra automaticky.

Pravidlo: `<service-name>:<port>`

Cez namespace: `<service-name>.<namespace>:<port>`

URL je v `fe-config.API_URL`, takže ju môžeš zmeniť bez rebuildu image-u.

Najčastejšia chyba pri 2 deploymentoch: zabudneš zmeniť selector v Service. Service exam-be-svc musí mať selector .app: exam-be, nie exam-app — inak BE service neposiela traffic do žiadneho podu.

8. Nasadenie a overenie

Krok za krokom

```
# 1. Postavíme image (lokálne do Minikube)
docker build -t exam-app:latest -f Dockerfile .
minikube image load exam-app:latest

# 2. Aplikujeme všetky manifesty
kubectl apply -f k8s/
#           ▲ aplikuje VŠETKY .yaml súbory v priečinku

# 3. Overíme stav
kubectl -n exam-meno get all,pvc
#           ▲ -n = namespace (bez tohto ti nič neukáže)
```

Cielový stav (čo musíš vidieť)

NAME	READY	STATUS	RESTARTS	
pod/exam-app-xxxxxxxxx-yyyyy	1/1	Running	0	← 1/1, Running

NAME	TYPE	CLUSTER-IP	PORT(S)	
service/exam-svc	NodePort	10.x.x.x	80:30080/TCP	← NodePort

NAME	READY	UP-TO-DATE	AVAILABLE	
deployment.apps/exam-app	1/1	1	1	← 1/1

NAME	STATUS	VOLUME	CAPACITY	
persistentvolumeclaim/exam-pvc	Bound	exam-pv	1Gi	← MUSÍ byť Bound

Otvor v prehliadači

```
minikube service exam-svc -n exam-meno
# alebo:
kubectl -n exam-meno port-forward svc/exam-svc 8080:80
# → http://localhost:8080
```

Dôkaz perzistencie (povinný pri obhajobe)

Toto musíš ukázať — že dáta prežijú smrť podu:

```
# 1. nahraj súbor / zmeň niečo cez UI
# 2. zabi pod
kubectl -n exam-meno delete pods --all

# 3. počkaj kým sa vytvorí nový (5-10 sekúnd)
kubectl -n exam-meno get pods -w
#           ▲ -w = watch (čaká na zmeny, Ctrl+C ukončí)

# 4. refreshni stránku – dáta sú stále tam = PVC funguje
```

9. Debug — keď niečo nejde

Príkazy v poradí, keď ti nefunguje

1. Vidím všetko v namespace?

```
kubectl -n exam-meno get all,pvc
```

2. Aký je stav podu?

```
kubectl -n exam-meno get pods
```

→ Pending = ešte sa nespustil (čakáme na PVC, image...)

→ ContainerCreating = ťahá image

→ CrashLoopBackOff = appka vnútri padá

→ ImagePullBackOff = nevie nájsť image

→ Running = OK

3. Detail podu (events na konci sú zlato)

```
kubectl -n exam-meno describe pod -l app=exam-app | tail -40
```

4. Logy aplikácie

```
kubectl -n exam-meno logs deploy/exam-app
```

▲ názov deployment-u

5. Logy InitContainer-a (keď máš)

```
kubectl -n exam-meno logs deploy/exam-app -c init-data
```

▲ názov initContainera

6. Dnu do podu (debugovanie)

```
kubectl -n exam-meno exec -it deploy/exam-app -- bash
```

7. Pozri obsah PVC z podu

```
kubectl -n exam-meno exec deploy/exam-app -- ls -la /var/www/html/data
```

8. Reštartuj pody (po zmene ConfigMap)

```
kubectl -n exam-meno rollout restart deployment/exam-app
```

Najčastejšie chyby

Symptóm	Čo to znamená	Fix
ImagePullBackOff	K8s ťahá image z internetu, nemá ho	minikube image load exam-app:latest + imagePullPolicy: IfNotPresent
pvc Pending	PVC sa neviaže na PV	storageClassName v PV a PVC musí byť rovnaké slovo
pod Pending: persistentvolumeclaim "X" not found	claimName v deployment ≠ názov PVC	Skontroluj claimName: exam-pvc v deployment.yaml
Permission denied (PHP nemôže písať)	Apache (UID 33) nie je vlastníkom priečinka	InitContainer musí robiť chown -R 33:33 /data
localhost:30080 neodpovedá	Docker driver Minikube nepublikuje NodePort	Použi minikube service alebo port-forward

Symptóm	Čo to znamená	Fix
port already allocated	nodePort je obsadený	Zmeň nodePort (rozsah 30000-32767)
Service neposiela traffic do podu	Selector Service ≠ Labels Deployment-u	Skontroluj že selector.app: v service = labels.app: v deployment
unknown field "spec.HostPath"	YAML je case-sensitive	hostPath (malé h), nie HostPath
Po zmene ConfigMap-y appka stále stará	Pody ju nepoznajú	kubectl rollout restart deployment/...
welcome.txt zatiaľ neexistuje v UI	InitContainer zlyhal	kubectl logs deploy/exam-app -c init-data

10. Upratanie

```
kubectl delete -f k8s/           # zmaže všetko z manifestov
kubectl delete pv exam-pv        # PV je cluster-scoped, treba zvlášť
docker rmi exam-app:latest       # zmaže image
```

11. Cheatsheet — len príkazy v poradí

```
# Príprava
mkdir -p ~/exam/{src,k8s}
cd ~/exam
# ... nakopíruj súbory zo šablóny ...

# Build
docker build -t exam-app:latest -f Dockerfile .
minikube image load exam-app:latest

# Nasadenie
kubectl apply -f k8s/

# Overenie
kubectl -n exam-meno get all,pvc

# Otvorenie
minikube service exam-svc -n exam-meno
# alebo
kubectl -n exam-meno port-forward svc/exam-svc 8080:80

# Test perzistencie
kubectl -n exam-meno delete pods --all
kubectl -n exam-meno get pods -w

# Upratanie
kubectl delete -f k8s/
kubectl delete pv exam-pv
```

12. Mapa: ktorá hodnota kde a s čím sa zhoduje

